FACULDADE DE TECNOLOGIA DE SÃO JOSÉ DOS CAMPOS

FATEC PROFESSOR JESSEN VIDAL

ANA BEATRIZ V. CHAGAS
CELIA CHAGAS FELIPE
JESSICA VITÓRIA M. REIS
LETICIA RAFAELA RIBEIRO
PATRICIA DO C. LOURENÇO
PEDRO DE VASCONCELOS RODRIGUES

TALITHA DE BARROS F. SANTOS

MANUAL DE TRATAMENTO DA BASE RAIS

São José dos Campos

2026

SUMÁRIO

1. INTRODUÇÃO 2
2. OBJETIVO 2
3. FERRAMENTAS UTILIZADAS 3
4. TRATAMENTO 3
4.2. Preparação para ambiente do Python 4
4.3. Leitura da base em blocos 4
4.4. Identificação de São José dos Campos 5
4.5. Filtragem de registros 6
4.5.1. Resultado da Filtragem 6
4.6. Limpeza dos dados 7
4.7. Padronização dos textos 7
4.8. Padronização dos valores ausentes 7
4.9. Remoção de registros completamente vazios 7
4.10. Remoção de duplicidades 8
4.11. Reorganização do índice 8
4.12. Validação da base tratada 9
4.13. Exportação para Excel 9
4.14. Resultado da primeira fase 9
4.15.1. Tratamento CNAE 2.0 10
4.15.2 Padronização da CNAE 11
4.16. Criação das dimensões 11
4.17. Estrutura para Power BI 11
4.18. Importação para POWER BI 11
4.19. Fluxo geral do tratamento 12
5. CONCLUSÃO 13

1. INTRODUÇÃO

Este manual apresenta o processo utilizado para tratamento da base de dados da Relação Anual de Informações Sociais (RAIS), desde a leitura do arquivo original até a geração da base preparada para utilização nas etapas posteriores do projeto.

Devido ao elevado volume de registros presentes no arquivo original, o tratamento foi realizado utilizando a linguagem Python, com auxílio da biblioteca Pandas e do ambiente Visual Studio Code.

O processo envolveu a leitura da base em blocos, identificação dos registros pertencentes ao município de São José dos Campos, limpeza e padronização dos dados, remoção de registros duplicados e exportação da base tratada.

Posteriormente, foram utilizadas tabelas auxiliares, como a estrutura da CNAE 2.0, permitindo complementar as informações da RAIS e preparar os dados para modelagem e análise no Power BI.

2. OBJETIVO

O principal objetivo do tratamento foi transformar a base bruta da RAIS em uma base menor, organizada e adequada às análises propostas no projeto.

O processo buscou:

- Selecionar somente registros referentes a São José dos Campos;
- Reduzir o volume de dados processados;
- Padronizar informações textuais;
- Identificar valores ausentes;
- Remover registros duplicados;
- Preparar códigos utilizados nos relacionamentos;
- Complementar os dados utilizando tabelas auxiliares;
- Gerar arquivos adequados para utilização no Power BI.

3. FERRAMENTAS UTILIZADAS

Figura 1 – Ferramentas.

Ferramentas	Utilização
Python	Desenvolvimento do tratamento
Pandas	Manipulação e limpeza dos dados
Visual Studio Code	Execução e desenvolvimento dos código
Excel	Conferência das bases resultantes
Power BI	Modelagem, relacionamento e visualização

Fonte: Fonte própria (2026).

4. TRATAMENTO

4.1 Base de dados original

```
RAIS_ESTAB_PUB.txt
```

A base possuía 24 colunas e aproximadamente 12.643.067 registros.

Como o volume de dados era muito elevado para trabalhar diretamente em ferramentas como Excel, optou-se pelo processamento utilizando Python. O arquivo possuía os dados separados pelo caractere:

```
;
```

e foi necessário utilizar a codificação:

```
latin1
```

para que caracteres especiais e acentuação fossem interpretados corretamente.

4.2. Preparação para ambiente do Python

A primeira etapa do código consistiu na importação da biblioteca Pandas:

```
import pandas as pd
```

O Pandas é a principal biblioteca utilizada no tratamento. Ele permite trabalhar com estruturas semelhantes a tabelas, chamadas DataFrames. Durante o projeto, ele foi utilizado para ler, filtrar, limpar, organizar, validar e exportar os dados da RAIS.

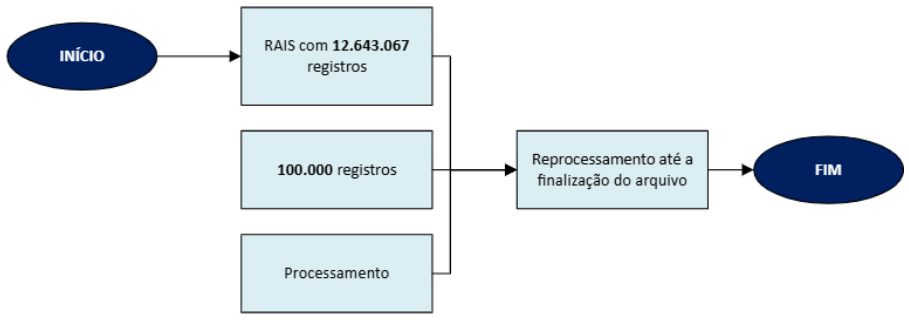
4.3. Leitura da base em blocos

O arquivo original possuía mais de 12 milhões de registros. Carregar toda a base simultaneamente poderia consumir uma quantidade elevada de memória do computador. Por isso, utilizamos o processamento em blocos de 100.000 registros.

```
for bloco in pd.read_csv(
    "RAIS_ESTAB_PUB.txt",
    sep=";",
    encoding="latin1",
    dtype=str,
    chunksize=100000
):
```

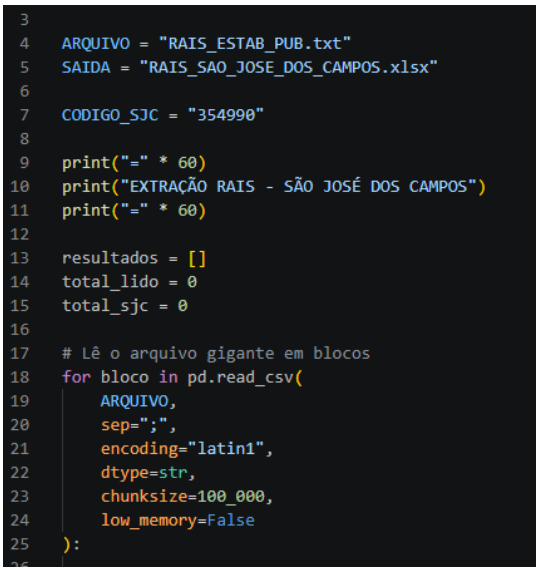
- **sep=";"** – Define que as colunas do arquivo são separadas por ponto e vírgula.
- **encoding="latin1"** – Define a codificação utilizada na leitura do arquivo
- **dtype=str** – Faz com que inicialmente todas as informações sejam interpretadas como texto. Isso é especialmente importante para códigos numéricos, pois evita alterações automáticas.
- **chunksize=100000** – Determina que o Pandas processe 100.000 registros por vez

Figura 2 - Fluxograma e tratamento.



Fonte: Fonte própria (2026).

Figura 3 - Código de leitura da base.



```
3
4   ARQUIVO = "RAIS_ESTAB_PUB.txt"
5   SAIDA = "RAIS_SAO_JOSE_DOS_CAMPOS.xlsx"
6
7   CODIGO_SJC = "354990"
8
9   print("=" * 60)
10  print("EXTRAÇÃO RAIS - SÃO JOSÉ DOS CAMPOS")
11  print("=" * 60)
12
13  resultados = []
14  total_lido = 0
15  total_sjc = 0
16
17  # Lê o arquivo gigante em blocos
18  for bloco in pd.read_csv(
19      ARQUIVO,
20      sep=";",
21      encoding="latin1",
22      dtype=str,
23      chunksize=100_000,
24      low_memory=False
25  ):
```

Fonte: Fonte própria (2026).

4.4. Identificação de São José dos Campos

Como o objetivo do projeto era estudar São José dos Campos, foi necessário localizar o município dentro da RAIS. Na base utilizada, o campo Município utilizava códigos com seis dígitos. O código identificado para São José dos Campos foi:

```
354990
```

Essa informação foi utilizada como critério para o filtro.

4.5. Filtragem de registros

Dentro de cada bloco de 100.000 registros, o Python verificava a coluna Município e selecionava somente os registros correspondentes a 354990.

```
filtro = bloco[
    bloco["Município"].str.strip() == "354990"
]
```

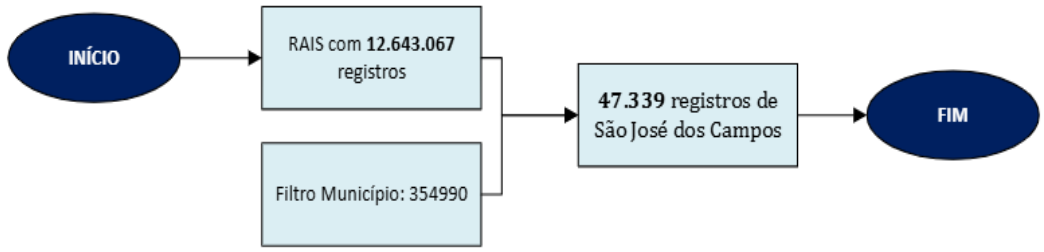
- **.str.strip()**= Esse comando remove espaços existentes antes ou depois do valor.

Isso evita que um registro correto seja ignorado simplesmente porque havia espaços no arquivo original.

4.5.1. Resultado da Filtragem

Após o Python percorrer os aproximadamente 12.643.067 registros, foram encontrados 47.339 registros correspondentes a São José dos Campos. Isso representou uma redução significativa da quantidade de dados que precisaria ser utilizada nas próximas etapas.

Figura 4 – Etapas da filtragem.



Fonte: Fonte própria (2026).

4.6. Limpeza dos dados

Depois de selecionar São José dos Campos, iniciamos a etapa de limpeza. Primeiramente, os valores ausentes foram temporariamente substituídos por texto vazio:

```
df = df.fillna("")
```

Facilitando a execução das operações de limpeza textual nas diferentes colunas.

4.7. Padronização dos textos

Depois, percorremos as colunas da base:

```
for coluna in df.columns:
    df[coluna] = df[coluna].astype(str).str.strip()
```

Primeiramente, astype(str) converte os valores para texto.

Depois, str.strip() remove espaços no início e no final Essa padronização reduz problemas posteriores de comparação e relacionamento.

4.8. Padronização dos valores ausentes

Após a limpeza dos textos, os campos vazios foram novamente identificados como valores ausentes.

```
df = df.replace("", pd.NA)
```

O Pandas utiliza:

```
<NA>
```

para representar a ausência de informação. Isso permite diferenciar corretamente um campo realmente vazio de um texto válido.

4.9. Remoção de registros completamente vazios

A próxima etapa foi:

```
df = df.dropna(how="all")
```

Esse comando elimina somente registros nos quais todas as colunas estão vazias. Uma linha que possui apenas uma informação ausente não é automaticamente excluída. Ela só é removida quando o registro inteiro não possui informações úteis.

4.10. Remoção de duplicidades

Uma das etapas mais importantes foi identificar registros repetidos. O comando utilizado foi:

```
df = df.drop_duplicates()
```

Ele compara as linhas da base e remove registros duplicados.

Figura 5 – Etapas de remoção de duplicadas.

Etapa	Registros
Após filtro de SJC	47.339
Duplicados removidos	4.977
Base após a limpeza	42.362

Fonte: Fonte própria (2026).

4.11. Reorganização do índice

Após excluir registros, os números das linhas do DataFrame poderiam ficar descontínuos. Por isso utilizamos:

```
df = df.reset_index(drop=True)
```

Exemplo – antes:

```
0
1
4
7
9
```

Depois:

```
0
1
2
3
4
```

O parâmetro drop=True impede que o índice antigo seja transformado em uma nova coluna.

4.12. Validação da base tratada

Depois da limpeza, a base pode ser validada utilizando comandos como:

```
print(df.shape)
```

O shape apresenta duas informações:

```
(linhas, colunas)
```

Também podem ser utilizados:

```
print("Quantidade de registros:", len(df))
print("Quantidade de colunas:", len(df.columns))
```

Ao final do processo de limpeza, a base utilizada no projeto possuía 42.362 registros válidos.

4.13. Exportação para Excel

Depois do tratamento, a base foi exportada para Excel. O arquivo gerado foi:

```
RAIS_SAO_JOSE_DOS_CAMPOS.xlsx
```

e a planilha:

```
RAIS_SJC
```

Para trabalhar com arquivos .xlsx, utilizamos o mecanismo:

```
openpyxl
```

A exportação segue a lógica:

```
df.to_excel(
    "RAIS_SAO_JOSE_DOS_CAMPOS.xlsx",
    sheet_name="RAIS_SJC",
    index=False,
    engine="openpyxl"
)
```

index=False = Para impedir que o índice interno do Pandas seja criado como uma coluna adicional no Excel.

4.14. Resultado da primeira fase

Neste ponto, concluímos a primeira grande etapa:

Figura 6 – Fluxo da primeira fase de tratamento da base.



Fonte: Fonte própria (2026).

Esse fluxo pode ser utilizado como síntese visual da primeira fase do tratamento.

4.15.1. Tratamento CNAE 2.0

Depois da RAIS, iniciamos uma segunda fase de preparação dos dados. A RAIS possui códigos referentes às atividades econômicas dos estabelecimentos. Entretanto, somente visualizar esses códigos dificultaria a interpretação das informações. Por isso foi utilizada a estrutura da Classificação Nacional de Atividades Econômicas – CNAE 2.0. O arquivo utilizado nessa etapa foi:

```
CNAE20_EstruturaDetalhada.xls
```

A partir dele, os códigos poderiam ser associados às respectivas descrições e níveis da atividade econômica.

4.15.2 Padronização da CNAE

Durante essa etapa, os códigos foram preparados para permitir a utilização dos diferentes níveis da CNAE. A estrutura trabalhada considerava informações como a Seção, Divisão, Grupo, Classe e Atividade. Isso permitiu analisar os dados tanto de maneira mais ampla quanto detalhada. Por exemplo, em vez de apresentar somente um código CNAE, o Power BI poderia apresentar a descrição correspondente à atividade econômica.

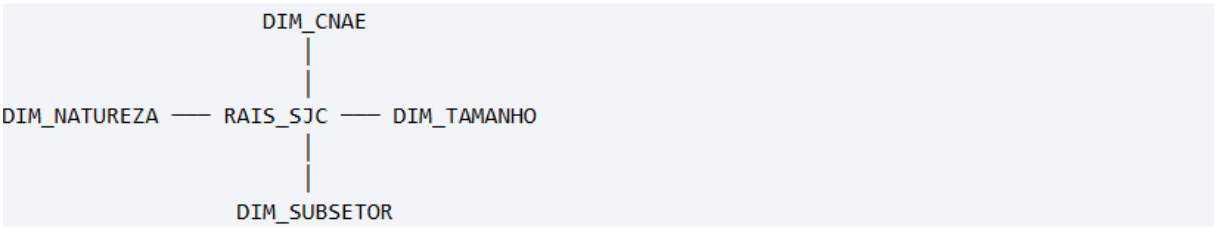
4.16. Criação das dimensões

Além da RAIS principal, foram preparadas tabelas auxiliares para organizar o modelo. Entre os arquivos do processo estavam:

```
CNAE_TRATADA.xlsx
DIM_CNAE.xlsx
DIM_SUBSETOR.xlsx
DIM_TAMANHO.xlsx
DIM_NATUREZA.xlsx
```

4.17. Estrutura para Power BI

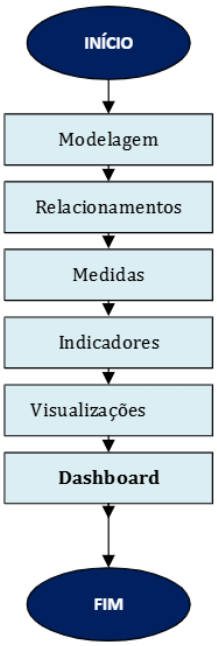
A organização passou a seguir uma lógica semelhante a:



4.18. Importação para POWER BI

Após a conclusão do tratamento, os arquivos puderam ser importados para o Power BI. A partir desse momento, a etapa deixou de ser somente de tratamento de dados e passou para:

Figura 7 – Fluxo de importação dos dados da base para o Power BI.

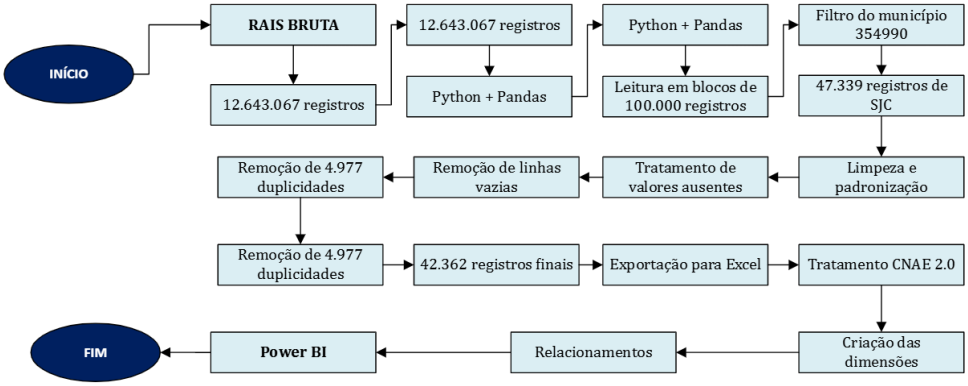


Fonte: Fonte própria (2026).

Essa separação demonstra que os dados não foram simplesmente colocados diretamente no Power BI. Primeiro houve uma etapa estruturada de preparação.

4.19. Fluxo geral do tratamento

Figura 8 – Fluxograma do tratamento da base.



Fonte: Fonte própria (2026).

Figura 9 – Erros encontrados e soluções.

Ocorrência	Correção aplicada
Importação abriu "Detalhamento"	Selecionar a aba "Resultado" no Power Query.
Erro "Ano inicial" não encontrado	Remover etapas duplicadas de Cabeçalhos Promovidos e Tipo Alterado, preservando as etapas válidas.
Todas as barras com mesmo tamanho	Trocar gráfico 100% empilhado por barras clusterizadas.
DAX não determina valor único de Codigo SH4	Criar Produto Resumido como Nova coluna, e não Nova medida.
"Produtos_Rosca" não encontrada	Criar uma Nova tabela independente e atualizar o nome referenciado na medida.
Rosca de importações mostrava exportações	Usar tabela auxiliar e medida específicas de IMPORTAÇÃO.

Fonte: Fonte própria (2026).

5. CONCLUSÃO

O tratamento realizado permitiu transformar a base original da RAIS, composta por mais de 12 milhões de registros, em uma base específica e estruturada para as necessidades do projeto.

A utilização do Python possibilitou processar o elevado volume de informações de maneira automatizada, utilizando a leitura em blocos para reduzir o consumo de recursos computacionais. Após a identificação dos registros pertencentes a São José dos Campos, foram realizadas etapas de limpeza, padronização e remoção de duplicidades.

Dos 47.339 registros inicialmente encontrados para o município, 4.977 registros duplicados foram removidos, resultando em uma base tratada com 42.362 registros.

Posteriormente, a utilização de tabelas auxiliares, especialmente a estrutura da CNAE 2.0, permitiu complementar e organizar as informações necessárias para a modelagem dos dados.

Dessa forma, o processo de tratamento criou uma base mais organizada, consistente e adequada para a construção das análises e visualizações desenvolvidas no Power BI.